

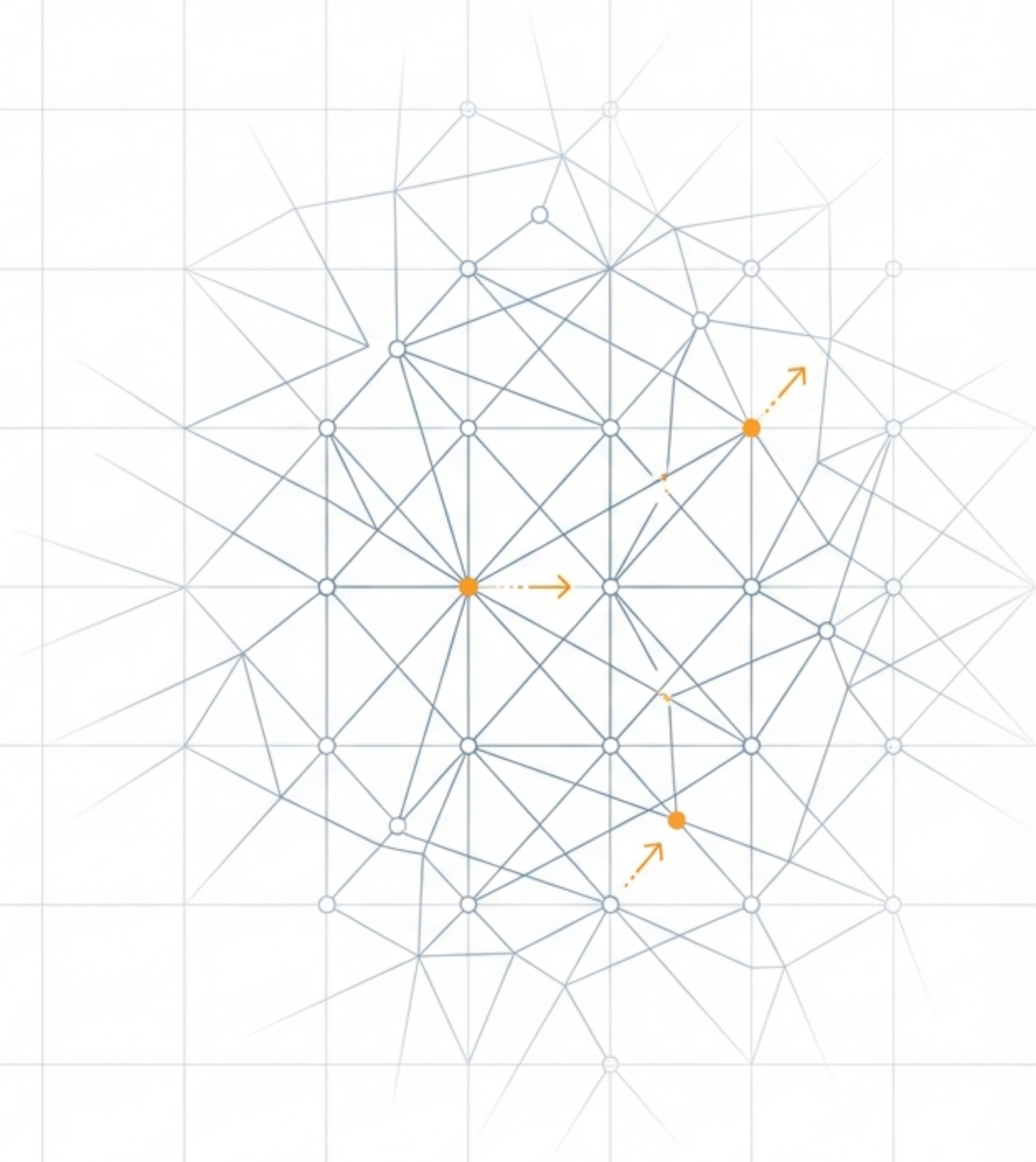
Web Content Filtering Proxy Solution

AWS Well-Architected Framework Review

DATE: OCTOBER 2023

STATUS: AUDIT COMPLETE

SCOPE: 6 PILLARS OF THE AWS WELL-ARCHITECTED FRAMEWORK



High-Performance Engineering Requires Operational Maturity to Match



The Win: Efficiency & Scale

The **stateless, serverless-first architecture** achieves a 9/10 rating for **Cost and Performance**.

The use of **Spot Instances** and "**Scale-to-Zero**" capability drives aggressive cost savings while maintaining global low latency.



The Risk: Governance & Safety

Operational Excellence (6/10) and **Security** (7/10) lag behind.

The current reliance on **GitHub Secrets** for credential injection and a lack of active **CloudWatch alarms** creates a significant "**blind spot**" in production environments.

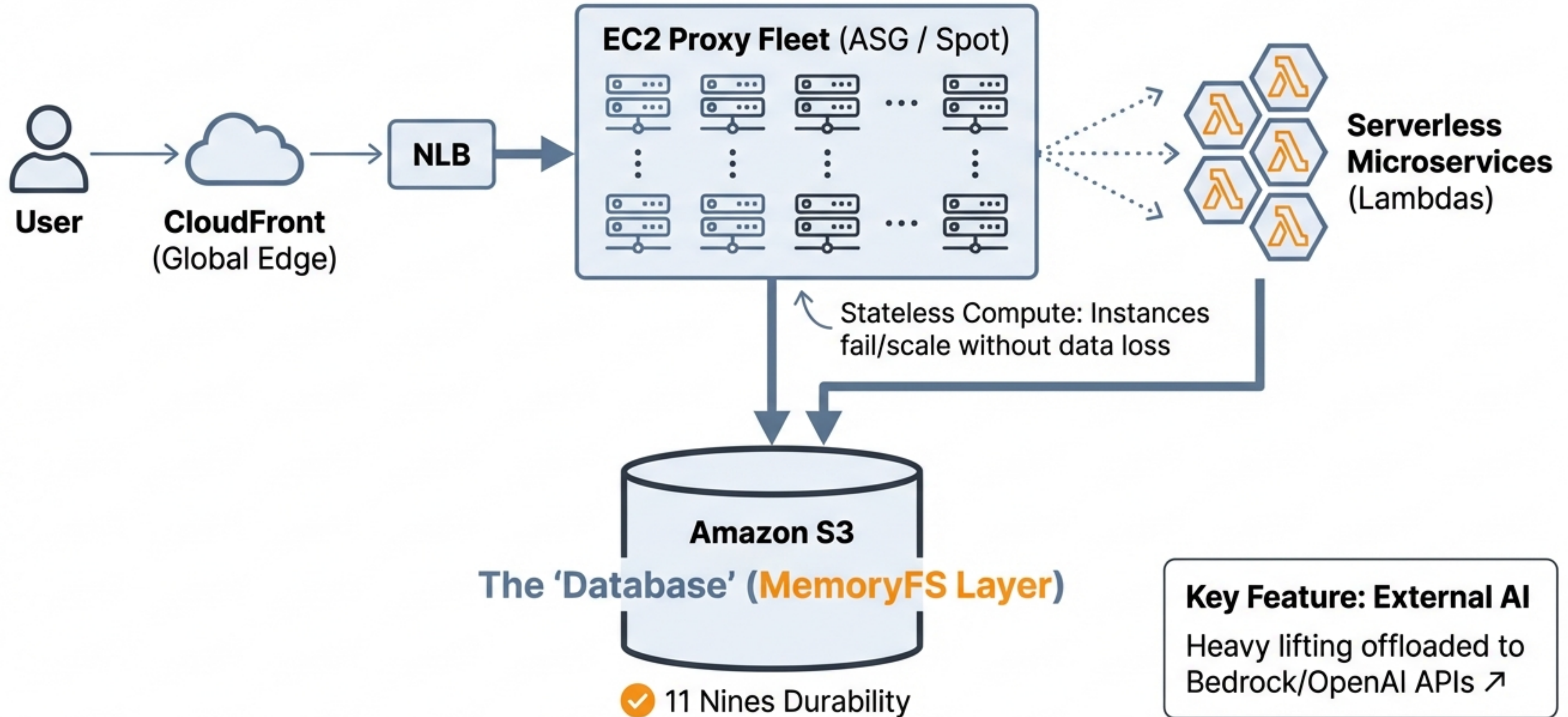


The Verdict: Solid Core, Needs Guardrails

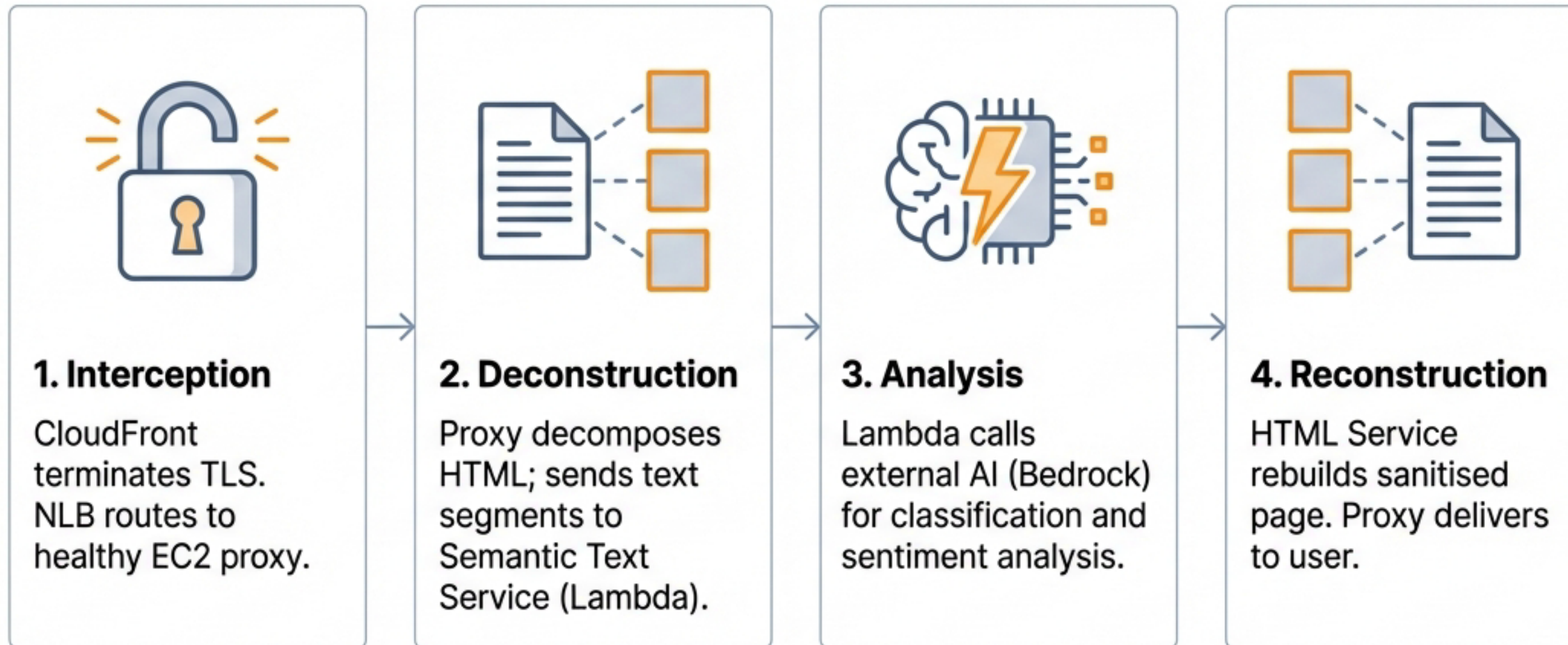
The code is **robust**; the governance is not.

Immediate **remediation** of **secrets management** and **monitoring** is required to move the solution from "**Proof of Concept**" quality to "**Enterprise Ready**".

A Stateless 'Man-in-the-Middle' Architecture

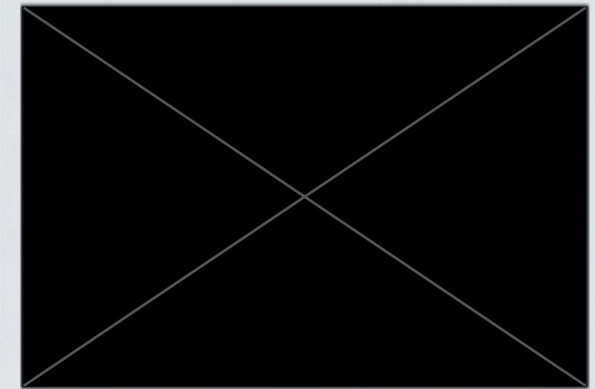


The Request Lifecycle: Deconstruct, Analyse, Reconstruct



Architectural Note

The 'S3 as Database' choice allows all components to share state (caching/logs) via a unified storage layer, eliminating the need for a traditional relational database.



Performance Efficiency: The Serverless Superpower

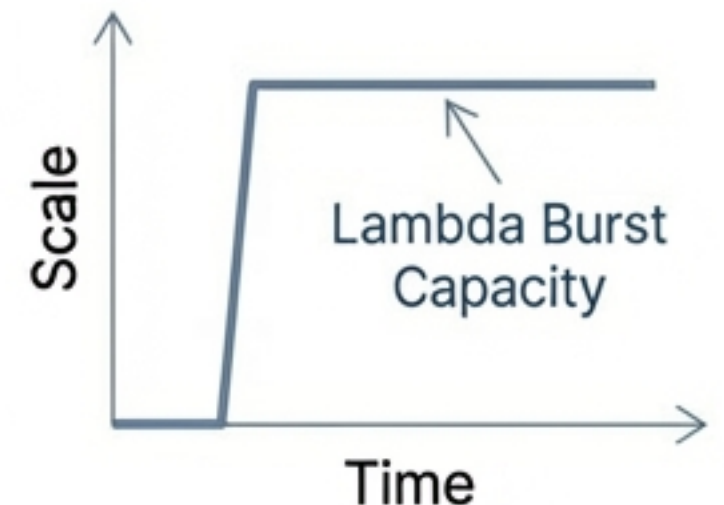
Well-Architected Rating: 9/10

Strengths

- **Scale to Zero:** Lambda-heavy design means zero resource contention during idle periods.
- **Global Edge:** CloudFront usage ensures low-latency TLS termination globally.
- **AI Offloading:** Democratising advanced tech by consuming managed services (Bedrock) rather than hosting heavy GPU clusters.

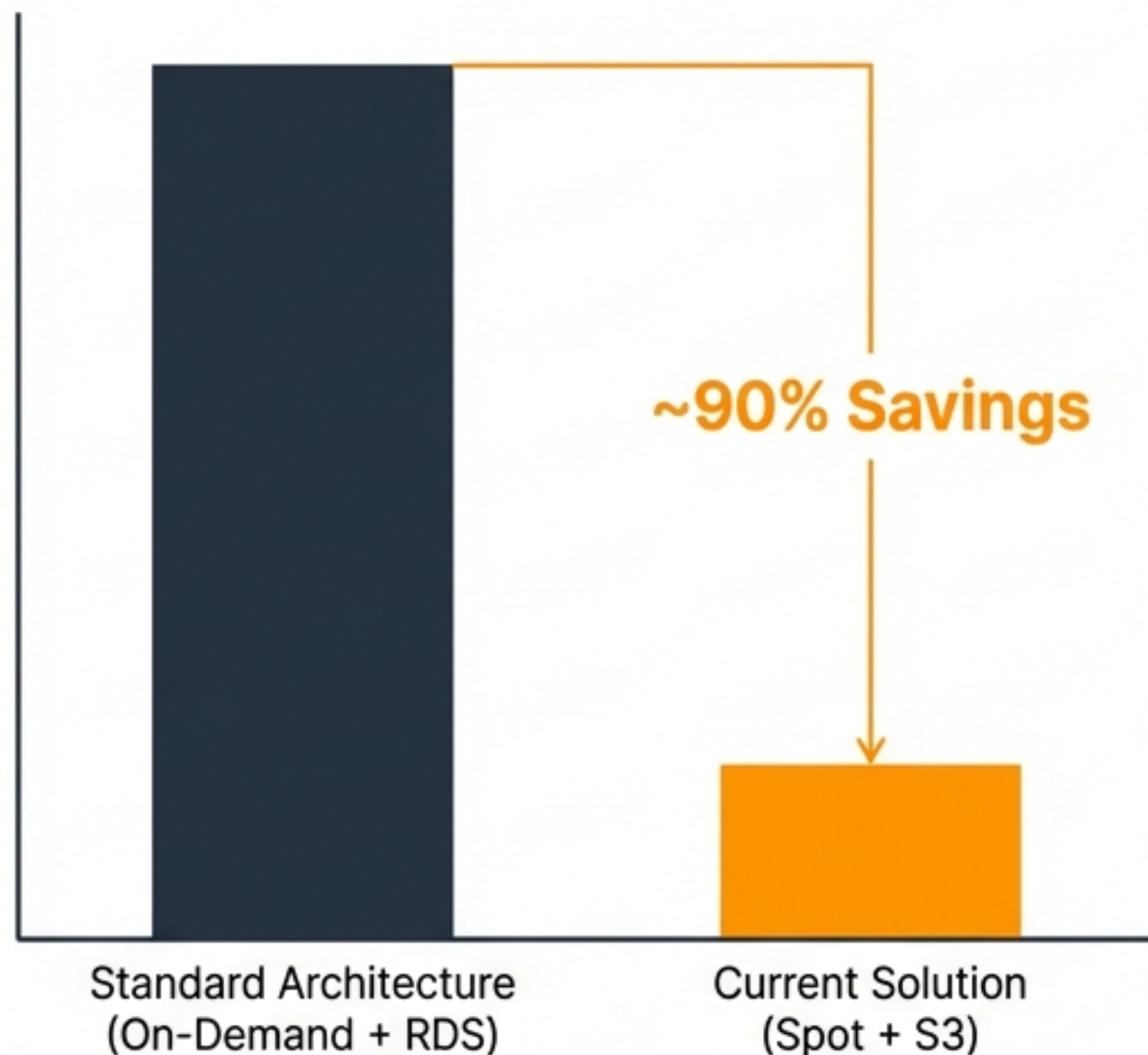
Optimisations

- **Cold Starts:** Monitor Lambda p95 latency; consider Provisioned Concurrency for critical paths.
- **S3 Latency:** Watch for read-after-write consistency delays; potential future need for Redis/ElastiCache if latency requirements tighten.



Cost Optimisation: Penny Pinching by Design

Well-Architected Rating: 9/10



- **Spot Strategy**

The stateless **EC2 proxies** leverage Spot Instances. The **Auto Scaling Group** handles interruptions gracefully.

- **Aggressive Teardown**

Environment capability to destroy the entire stack (excluding S3 data) after business hours eliminates idle costs.

- **S3 Economics**

Using S3 instead of RDS/DynamoDB drastically lowers the monthly storage burn rate.

“Matching energy usage and cost to actual demand.”

Reliability: Bending Without Breaking

Well-Architected Rating: 8/10



Multi-AZ Resilience.

ASG spans multiple zones; NLB health checks automatically reroute traffic away from failed instances.



11 Nines of Durability.

Relying on S3 with Versioning inherits AWS's highest durability tier. Data is inherently multi-AZ.



Immutability.

CI/CD builds immutable AMIs. Rollbacks are clean and reliable.

The Gap

Lack of formal 'Game Days'. The architecture is resilient in theory, but failover scenarios (e.g., AZ outages) have not been physically simulated.

Sustainability: Efficiency is Green

Well-Architected Rating: 8/10



- **Scale to Zero**

In the cloud, sustainability is a proxy for utilisation. The system's 'Scale to Zero' capability ensures no energy is wasted on idle servers.

- **Shared Responsibility**

Leveraging AWS managed services (Lambda/CloudFront) benefits from AWS's economy of scale and renewable energy projects.

- **Future Opportunity:
Graviton Migration**

Transitioning the EC2 fleet to ARM-based Graviton instances (e.g., C7g) could yield better performance-per-watt and further cost reductions.

Security: Strong Perimeter, Soft Centre

Well-Architected Rating: 7/10 (Priority Focus)

- **The Good:**

Strong IAM Scoping: Least Privilege applied to most roles.

Encryption: TLS encryption in transit everywhere.



- **The Critical Gap:**

Secrets Management Failure

Current State: API keys injected via GitHub Secrets into plaintext environment variables.

Risk: Secrets visible in console; rotation is manual.

Required Fix: Migrate to **AWS Secrets Manager**. Enable runtime retrieval and automated rotation.

Secondary Gap: Lack of GuardDuty/CloudTrail active threat detection.

Operational Excellence: Flying Blind

Well-Architected Rating: 6/10 (Priority Focus)



Automation: CI/CD pipelines are robust; infrastructure is codified (IaC).



Visibility: No CloudWatch Dashboards or active Alarms. The team relies on reactive user reports rather than proactive metrics.



Process: No Runbooks or Playbooks exist. Knowledge resides in developers' heads, creating a 'Bus Factor' risk.

Impact: Without alarms on '**Lambda Throttles**' or '**5xx Errors**', minor issues will escalate into outages before the team notices.

The Well-Architected Scorecard



The solution is a high-performance engine lacking the necessary **dashboard** and **safety belts**. The **architecture** is sound; the **operational wrapper** needs maturity.

Remediation Roadmap: From Audit to Action

Horizons

NOW (Horizon 1)

Immediate Remediation

- **Security:** Implement [AWS Secrets Manager](#); remove secrets from [GitHub](#).
- **Ops:** Define critical [CloudWatch Alarms](#) (Error Rates, Latency) and create a basic [System Health Dashboard](#).

NEXT (Horizon 2)

Operational Maturity

- **Process:** Document [Runbooks](#) for common failures (e.g., "[Proxy Unhealthy](#)").
- **Drills:** Conduct the first "[Game Day](#)" (simulate an [AZ failure](#)).

LATER (Horizon 3)

Optimisation

- **Resilience:** Pilot "[Active-Active](#)" [Multi-Region](#) deployment.
- **Efficiency:** Migrate [EC2](#) fleet to [Graviton](#) processors.